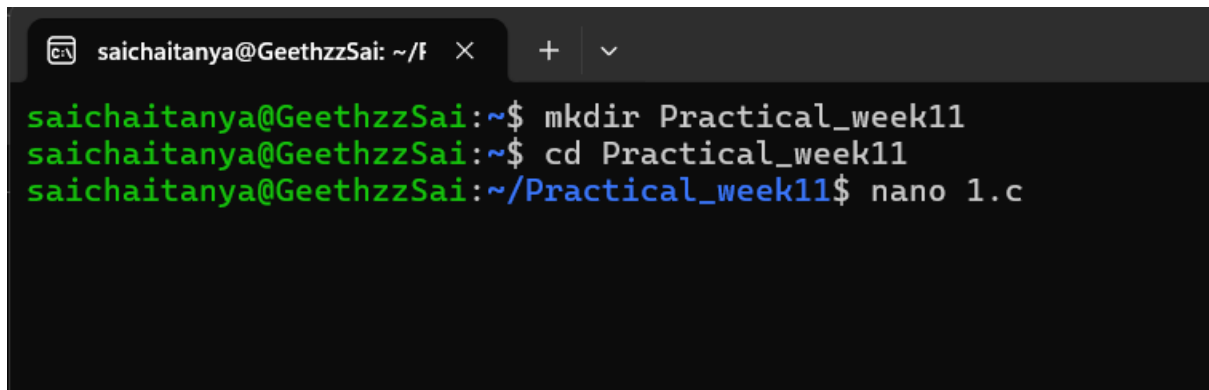

PRACTICAL WEEK 11

Task 1: Develop a multithreaded counter application using POSIX threads (pthread create, pthread_join). Demonstrate race conditions by allowing multiple threads to update a shared variable concurrently.

1) Create a file

A terminal window with a dark background and light green text. The window title is 'saichaitanya@GeethzzSai: ~/f'. The terminal shows three commands being executed: 'mkdir Practical_week11', 'cd Practical_week11', and 'nano 1.c'. The prompt changes from '~\$' to '~/Practical_week11\$' after the directory change.

```
saichaitanya@GeethzzSai:~/f$ mkdir Practical_week11
saichaitanya@GeethzzSai:~/f$ cd Practical_week11
saichaitanya@GeethzzSai:~/Practical_week11$ nano 1.c
```

Then press enter to open command

2) write a code

```
saichaitanya@GeethzzSai: ~/f × + ▾
GNU nano 8.7.1 1.c *
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

#define THREADS 4
#define COUNT 1000

int counter = 0;

void *increment(void *arg)
{
    for (int i = 0; i < COUNT; i++) {
        int temp = counter;
        usleep(1); // makes race condition easier to observe
        counter = temp + 1;
    }
    return NULL;
}

int main()
{
    pthread_t t[THREADS];

    for (int i = 0; i < THREADS; i++)
        pthread_create(&t[i], NULL, increment, NULL);

    for (int i = 0; i < THREADS; i++)
        pthread_join(t[i], NULL);

    printf("Expected counter = %d\n", THREADS * COUNT);
    printf("Actual counter = %d\n", counter);

    return 0;
}
```

```
saichaitanya@GeethzzSai: ~/f × + ▾
saichaitanya@GeethzzSai:~$ mkdir Practical_week11
saichaitanya@GeethzzSai:~$ cd Practical_week11
saichaitanya@GeethzzSai:~/Practical_week11$ nano 1.c
saichaitanya@GeethzzSai:~/Practical_week11$ gcc 1.c -o 1
saichaitanya@GeethzzSai:~/Practical_week11$ ./1
```

OUTPUT:

```
saichaitanya@GeethzzSai: ~/f × + v
saichaitanya@GeethzzSai:~$ mkdir Practical_week11
saichaitanya@GeethzzSai:~$ cd Practical_week11
saichaitanya@GeethzzSai:~/Practical_week11$ nano 1.c
saichaitanya@GeethzzSai:~/Practical_week11$ gcc 1.c -o 1
saichaitanya@GeethzzSai:~/Practical_week11$ ./1
Expected counter = 4000
Actual counter   = 1001
saichaitanya@GeethzzSai:~/Practical_week11$ |
```

Task-2: Modify the previous program using mutex locks (pthread mutex lock, pthread mutex_unlock) and compare the outputs with the race- condition version. Analyze the impact of synchronization.

CODE:

```
#include <stdio.h>
#include <pthread.h>

#define THREADS 4
#define COUNT 1000

int counter = 0;
pthread_mutex_t lock;

void *increment(void *arg)
{
    for (int i = 0; i < COUNT; i++) {
        pthread_mutex_lock(&lock);

        counter++;

        pthread_mutex_unlock(&lock);
    }
    return NULL;
}

int main()
{
    pthread_t t[THREADS];

    pthread_mutex_init(&lock, NULL);

    for (int i = 0; i < THREADS; i++)
        pthread_create(&t[i], NULL, increment, NULL);

    for (int i = 0; i < THREADS; i++)
        pthread_join(t[i], NULL);

    printf("Expected counter = %d\n", THREADS * COUNT);
    printf("Actual counter    = %d\n", counter);

    pthread_mutex_destroy(&lock);
}
```

```

int main()
{
    pthread_t t[THREADS];

    pthread_mutex_init(&lock, NULL);

    for (int i = 0; i < THREADS; i++)
        pthread_create(&t[i], NULL, increment, NULL);

    for (int i = 0; i < THREADS; i++)
        pthread_join(t[i], NULL);

    printf("Expected counter = %d\n", THREADS * COUNT);
    printf("Actual counter   = %d\n", counter);

    pthread_mutex_destroy(&lock);

    return 0;
}

```

OUTPUT:

```

saichaitanya@GeethzzSai:~/Practical_week11$
saichaitanya@GeethzzSai:~/Practical_week11$
saichaitanya@GeethzzSai:~/Practical_week11$ nano 2.c
saichaitanya@GeethzzSai:~/Practical_week11$ gcc 2.c -o 2
saichaitanya@GeethzzSai:~/Practical_week11$ ./2
Expected counter = 4000
Actual counter   = 4000
saichaitanya@GeethzzSai:~/Practical_week11$ |

```